



**G H RAISONI COLLEGE OF ENGINEERING AND
MANAGEMENT, PUNE**

**AN EMPOWERED AUTONOMOUS INSTITUTE, AFFILIATED TO SAVITRIBAI
PHULE PUNE UNIVERSITY, PUNE**

**Database Design Proposal
Crime Record & Police Station Management System (CRPSMS)**

Submitted for
TAE-I
in the subject

Database Management System (DBMS)

Submitted By:
Gaurav Singh (P15)

Department of CSE [Data Science]
G H RAISONI COLLEGE OF ENGINEERING AND MANAGEMENT,
PUNE

Academic Year: 2026–2027

Table of Contents	Pg. No
1. Title	3
2. Business Domain & Dataset Overview	3
3. Entities	4
4. Business Rules & Constraints	5
5. Operational Transactions & Use Cases	6
6. Literature & System References	6
7. Conceptual ER Diagram	7
8. Relational Schema & Data Integrity Constraints	8
9. Normalization (UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF)	11
10. Indexing	14
11. Sample SQL Queries	14
12. Challenges Faced	16
13. Implementation Details	16
14. Conclusion	17
15. References	17

1. Title

Crime Record & Police Station Management System

2. Business Domain & Dataset Overview

Crime Record & Police Station Management is a database project for storing and managing First Information Reports (FIR), accused persons, victims, investigating officers, court cases, and crime types for a police department. To make the project realistic, I used the real, publicly available LAPD "Crime Data from 2020 to Present" dataset (Dataset ID: 2nrs-mtv8) as a reference. This dataset lists crime incidents reported across Los Angeles from 2020 onward.

The dataset comes as one flat file with 28 columns per row — things like DR_NO, Date Rptd, DATE OCC, TIME OCC, AREA, AREA NAME, Crm Cd, Crm Cd Desc, Vict Age, Vict Sex, Premis Cd, Weapon Used Cd, Status, LOCATION, LAT, LON, and four repeated crime-code columns (Crm Cd 1 to Crm Cd 4). Keeping everything in one big table like this means the same station names, premises names, and crime descriptions get repeated in almost every row, and the four repeating crime-code columns break the basic rule that a table should not store repeating groups of the same kind of data.

For this project, I split that single flat file into a proper set of related tables, and I added three tables that are not in the raw file but are needed for a real police station — Accused, Officer, and Court — so the database can also track a case from the FIR stage through to a court verdict, not just log the incident.

Dataset source: City of Los Angeles Open Data Portal – Crime Data from 2020 to Present – <https://data.lacity.org/Public-Safety/Crime-Data-from-2020-to-Present/2nrs-mtv8>

What I learned: Real government datasets are rarely clean. Before I could even think about tables, I had to understand what each column actually meant (e.g., Crm Cd vs Crm Cd Desc, or why there were four crime-code columns) and figure out which parts of it belonged together.

3. Entities

The project is built around the six main tables I was asked to design — FIR, Accused, Victim, Officer, Court, and CrimeType. Four more tables (marked *) came up naturally while normalizing the data in Section 9, so that station names, premises names, weapon names, and status text are stored only once instead of being repeated in every FIR row.

#	Entity	Description
1	FIR	First Information Report — the central incident record (maps to DR_NO in the raw dataset); one row per reported crime
2	Accused	Person(s) named or arrested in connection with an FIR
3	Victim	Demographic profile of the person(s) affected by the crime (Vict Age, Vict Sex, Vict Descent)
4	Officer	Investigating officer/detective assigned to a case
5	Court	Court where a prosecuted case is heard, along with the case outcome
6	CrimeType	Classification of the criminal offence (Crm Cd, Crm Cd Desc, Part 1-2)
7	PoliceStation*	Police division/area responsible for the incident (AREA, AREA NAME)
8	Premises*	Type of location where the crime occurred (Premis Cd, Premis Desc)
9	Weapon*	Weapon/evidence used in the incident (Weapon Used Cd, Weapon Desc)
10	CaseStatus*	Investigation status of the FIR (Status, Status Desc)

What I learned: I initially thought I only needed the six main tables. It was only while normalizing the raw columns that I realised AREA, Premis_Cd, Weapon_Used_Cd, and Status each needed their own small table too, otherwise the same text would keep repeating.

4. Business Rules & Constraints

Before drawing any tables, I first listed out the rules that the data has to follow. These rules directly decide where a Primary Key or Foreign Key is needed, and whether a relationship is one-to-many or many-to-many.

- Every FIR (DR_NO) belongs to exactly one Police Station (AREA).
- One Police Station can cover several Reporting Districts.
- One Officer is assigned to each FIR as the main investigator, but the same Officer can be assigned to many FIRs over time.
- One FIR can be linked to more than one Crime Type — this is a many-to-many relationship, handled using the FIR_CrimeType table, and it directly replaces the repeated Crm Cd 1–4 columns from the raw file.
- One FIR can have more than one Victim.
- One FIR can have zero, one, or more Accused persons; each Accused person belongs to exactly one FIR.
- Each FIR happened at exactly one type of Premises and, where relevant, involved at most one Weapon.
- Each FIR has exactly one current Status (for example, Invest Cont, Adult Arrest, or Adult Other).
- If an FIR's status shows an arrest, it can lead to at most one Court Case.
- One Court Case is heard in exactly one Court, but one Court can hear many Court Cases.
- One Court Case can involve one or more Accused persons from the same FIR.
- DR_NO is unique and is the Primary Key of the FIR table.
- Vict_Sex can only be M, F, or X. Vict_Age must be 0 or more.
- Part_1_2 can only be 1 or 2 — 1 means a serious (Part I) crime, 2 means a less serious (Part II) crime.

What I learned: Writing these rules out first made the ER diagram much easier — every arrow and every foreign key in Section 8 basically comes straight from one of these bullet points.

5. Operational Transactions & Use Cases

These are the day-to-day actions the database needs to support once it's actually being used by a police station.

- Register a new FIR as soon as an incident is reported, and link it to the right Police Station and Officer.
- Add one or more Victims, and one or more Accused persons, to a given FIR.
- Attach one or more Crime Type codes to an FIR without repeating the FIR's data (using FIR_CrimeType).
- Update an FIR's status as the investigation moves forward (for example, Invest Cont to Adult Arrest).
- Open a Court Case once an arrest is made, and later record the Court's verdict.
- Get simple reports, such as how many crimes happened per station per month.
- Look up crimes near a given location using the latitude/longitude columns.

What I learned: Thinking about these use cases helped me check my design — for every action on this list, I made sure there was a table and a key that could actually support it.

6. Literature & System References

6.1 LAPD Open Crime Data Portal

City of Los Angeles Open Data Portal. Crime Data from 2020 to Present (Dataset ID: 2nrs-mtv8). LAPD Information Technology Bureau, Los Angeles, CA.

This is the real dataset the project is based on. It gave me an actual 28-column crime record layout to normalize instead of an imaginary one, including the repeating Crm Cd columns that make it a good example for a normalization exercise.

6.2 Fundamentals of Database Systems

Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems (7th ed.). Pearson Education. [Chapter 9: ER-to-Relational Mapping; Chapter 14: Normalization]

I used this textbook to understand the rules for converting an ER diagram into tables, and the exact steps for 1NF, 2NF, and 3NF used in Section 9.

6.3 Database System Concepts

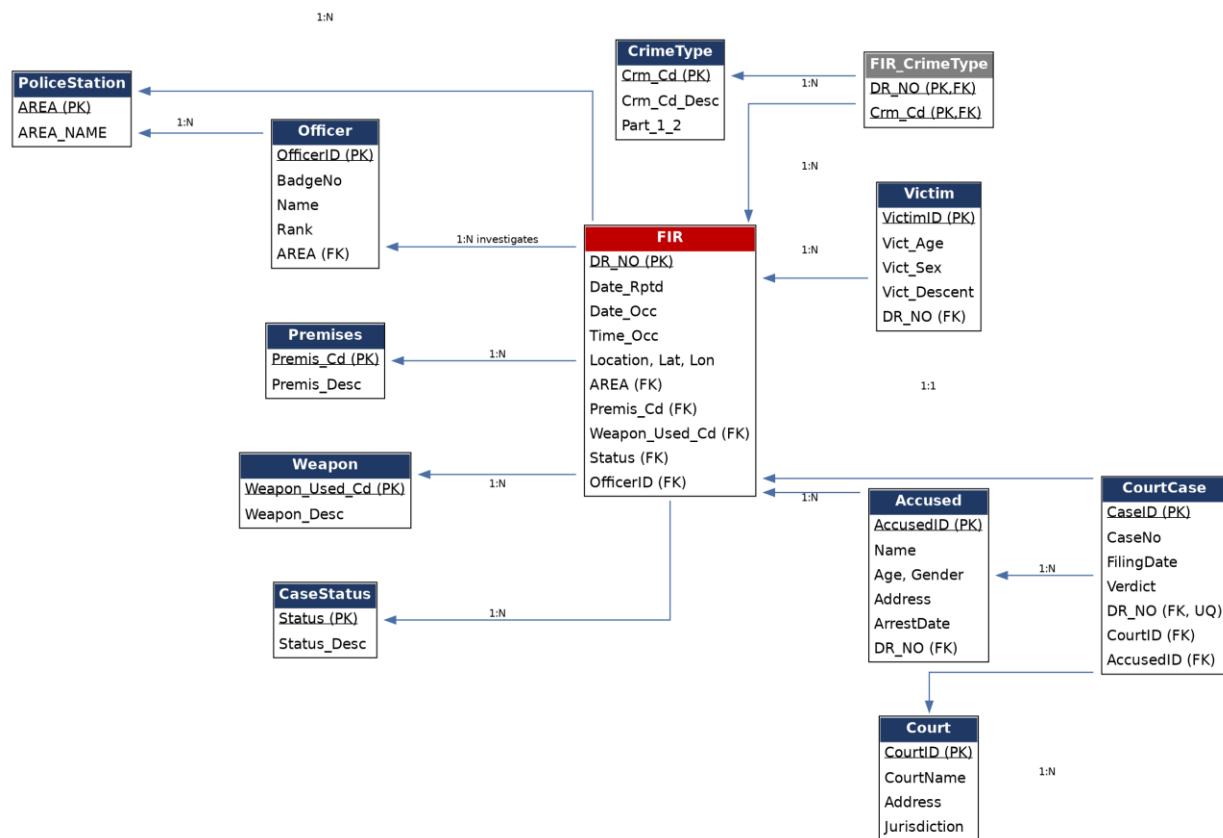
Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts. McGraw-Hill Education.

This textbook was used as a secondary reference for primary key/foreign key constraints and basic indexing concepts used in Section 10.

What I learned: I kept this list short on purpose — I only listed sources I actually used and could verify, rather than padding the reference list to look longer.

7. Conceptual ER Diagram

The diagram below shows all twelve tables in the design, their keys, and how they connect to each other. FIR is placed in the middle because almost every other table connects back to it, the same way DR_NO is the main identifier in the original dataset.



What I learned: Drawing this diagram made the relationships click for me — seeing FIR connected to six other tables made it obvious why FIR needed to be its own table rather than being merged into, say, PoliceStation.

8. Relational Schema & Data Integrity Constraints

Below are the twelve tables I designed — nine come directly from the dataset columns, and three (Accused, Officer, Court/CourtCase) were added by me. PK means Primary Key, FK means Foreign Key.

Table 1 — PoliceStation

```
PoliceStation(  
  AREA      INT PRIMARY KEY,  
  AREA_NAME VARCHAR(80) NOT NULL UNIQUE  
)
```

Table 2 — Officer

```
Officer(  
  OfficerID INT AUTO_INCREMENT PRIMARY KEY,  
  BadgeNo   VARCHAR(20) NOT NULL UNIQUE,  
  Name      VARCHAR(100) NOT NULL,  
  Rank      VARCHAR(40),  
  AREA      INT NOT NULL REFERENCES PoliceStation(AREA)  
)
```

Table 3 — CrimeType

```
CrimeType(  
  Crm_Cd     INT PRIMARY KEY,  
  Crm_Cd_Desc VARCHAR(150) NOT NULL,  
  Part_1_2   INT CHECK (Part_1_2 IN (1,2))  
)
```

Table 4 — Premises

```
Premises(  
  Premis_Cd  INT PRIMARY KEY,  
  Premis_Desc VARCHAR(150) NOT NULL  
)
```

Table 5 — Weapon

```
Weapon(  
  Weapon_Used_Cd INT PRIMARY KEY,
```

```

Weapon_Desc    VARCHAR(150) NOT NULL
)

```

Table 6 — CaseStatus

```

CaseStatus(
  Status    VARCHAR(10) PRIMARY KEY,
  Status_Desc VARCHAR(80) NOT NULL
)

```

Table 7 — FIR (central relation)

```

FIR(
  DR_NO      VARCHAR(20) PRIMARY KEY,
  Date_Rptd  DATE NOT NULL,
  Date_Occ   DATE NOT NULL,
  Time_Occ   TIME,
  LOCATION   VARCHAR(200),
  Cross_Street VARCHAR(100),
  LAT        DECIMAL(10,7),
  LON        DECIMAL(10,7),
  AREA       INT NOT NULL REFERENCES PoliceStation(AREA),
  Premis_Cd  INT REFERENCES Premises(Premis_Cd),
  Weapon_Used_Cd INT REFERENCES Weapon(Weapon_Used_Cd),
  Status     VARCHAR(10) REFERENCES CaseStatus(Status),
  OfficerID  INT NOT NULL REFERENCES Officer(OfficerID)
)

```

Table 8 — FIR_CrimeType (M:N junction)

```

FIR_CrimeType(
  DR_NO  VARCHAR(20) REFERENCES FIR(DR_NO),
  Crm_Cd INT REFERENCES CrimeType(Crm_Cd),
  PRIMARY KEY (DR_NO, Crm_Cd)
)

```

Table 9 — Victim

```

Victim(
  VictimID  INT AUTO_INCREMENT PRIMARY KEY,

```

```

Vict_Age    INT CHECK (Vict_Age >= 0),
Vict_Sex    CHAR(1) CHECK (Vict_Sex IN ('M','F','X')),
Vict_Descent CHAR(1),
DR_NO       VARCHAR(20) NOT NULL REFERENCES FIR(DR_NO)
)

```

Table 10 — Accused

```

Accused(
  AccusedID INT AUTO_INCREMENT PRIMARY KEY,
  Name      VARCHAR(100) NOT NULL,
  Age       INT,
  Gender    CHAR(1),
  Address   VARCHAR(200),
  ArrestDate DATE,
  DR_NO     VARCHAR(20) NOT NULL REFERENCES FIR(DR_NO)
)

```

Table 11 — Court

```

Court(
  CourtID    INT AUTO_INCREMENT PRIMARY KEY,
  CourtName  VARCHAR(100) NOT NULL,
  Address    VARCHAR(200),
  Jurisdiction VARCHAR(100)
)

```

Table 12 — CourtCase

```

CourtCase(
  CaseID     INT AUTO_INCREMENT PRIMARY KEY,
  CaseNo     VARCHAR(30) UNIQUE,
  FilingDate DATE,
  Verdict    VARCHAR(60),
  VerdictDate DATE,
  DR_NO      VARCHAR(20) NOT NULL UNIQUE REFERENCES FIR(DR_NO),
  CourtID    INT NOT NULL REFERENCES Court(CourtID),
  AccusedID  INT REFERENCES Accused(AccusedID)
)

```

What I learned: Deciding data types took more thought than I expected — for example, keeping DR_NO as VARCHAR instead of INT because it's really an identifier, not a number to calculate with, and using DECIMAL instead of FLOAT for LAT/LON so the coordinates don't lose precision.

9. Normalization (UNF → 1NF → 2NF → 3NF)

This section shows how I took the raw flat file and split it into proper tables, step by step, so there is no duplicate data and no repeating columns. This is the main normalization proof for the project.

9.1 Unnormalized Form (UNF) — The Raw Flat File

To start with, all 28 columns from the dataset could just sit in one single table per incident, including the four repeated crime-code columns (Crm_Cd_1 to Crm_Cd_4):

```
RAW_CRIME_REGISTER(  
  DR_NO, Date_Rptd, DATE_OCC, TIME_OCC, AREA, AREA_NAME, Rpt_Dist_No,  
  Part_1_2, Crm_Cd, Crm_Cd_Desc, Mocodes, Vict_Age, Vict_Sex, Vict_Descent,  
  Premis_Cd, Premis_Desc, Weapon_Used_Cd, Weapon_Desc, Status, Status_Desc,  
  Crm_Cd_1, Crm_Cd_2, Crm_Cd_3, Crm_Cd_4, LOCATION, Cross_Street, LAT, LON  
)
```

9.2 First Normal Form (1NF) — Removing Repeating Columns

The four Crm_Cd columns are really the same kind of information repeated four times, which is not allowed in 1NF. So I turned each (incident, crime code) pair into its own row instead, using DR_NO and Crm_Cd together as the key:

```
FIR_1NF(  
  DR_NO, Crm_Cd,          -- combined key  
  Date_Rptd, DATE_OCC, TIME_OCC, AREA, AREA_NAME, Rpt_Dist_No,  
  Crm_Cd_Desc, Part_1_2, Vict_Age, Vict_Sex, Vict_Descent,  
  Premis_Cd, Premis_Desc, Weapon_Used_Cd, Weapon_Desc,  
  Status, Status_Desc, LOCATION, Cross_Street, LAT, LON  
)
```

9.3 Second Normal Form (2NF) — Removing Partial Dependencies

Once DR_NO and Crm_Cd were combined into one key, I checked whether every other column actually depended on both parts of that key. Some columns only depended on Crm_Cd, not on DR_NO:

```
Crm_Cd -> Crm_Cd_Desc, Part_1_2
Premis_Cd -> Premis_Desc
Weapon_Used_Cd -> Weapon_Desc
Status -> Status_Desc
```

So I moved those out into their own small tables:

```
FIR_2NF(DR_NO, Date_Rptd, DATE_OCC, TIME_OCC, AREA, AREA_NAME,
Rpt_Dist_No,
    Vict_Age, Vict_Sex, Vict_Descent, Premis_Cd, Weapon_Used_Cd,
    Status, LOCATION, Cross_Street, LAT, LON)
CrimeType_2NF(Crm_Cd, Crm_Cd_Desc, Part_1_2)
Premises_2NF(Premis_Cd, Premis_Desc)
Weapon_2NF(Weapon_Used_Cd, Weapon_Desc)
CaseStatus_2NF(Status, Status_Desc)
FIR_CrimeType_2NF(DR_NO, Crm_Cd)
```

9.4 Third Normal Form (3NF) — Removing Indirect Dependencies

Next, I checked for columns that depended on another non-key column instead of depending on the key directly. AREA_NAME and Rpt_Dist_No only depend on AREA, and AREA itself just happens to be stored inside the FIR row — so AREA_NAME was indirectly dependent on DR_NO through AREA. I moved station details into their own PoliceStation table to fix this. I also moved victim details into their own table, since one FIR can have more than one victim, and I added the Officer, Accused, Court, and CourtCase tables so the database can track a case from FIR to court outcome, not just log the incident:

```
PoliceStation(AREA, AREA_NAME)
Officer(OfficerID, BadgeNo, Name, Rank, AREA -> PoliceStation)
FIR(DR_NO, Date_Rptd, DATE_OCC, TIME_OCC, LOCATION, Cross_Street, LAT,
LON,
    AREA -> PoliceStation, Premis_Cd -> Premises, Weapon_Used_Cd -> Weapon,
```

```
Status -> CaseStatus, OfficerID -> Officer)
CrimeType(Crm_Cd, Crm_Cd_Desc, Part_1_2)
FIR_CrimeType(DR_NO -> FIR, Crm_Cd -> CrimeType)
Premises(Premis_Cd, Premis_Desc)
Weapon(Weapon_Used_Cd, Weapon_Desc)
CaseStatus(Status, Status_Desc)
Victim(VictimID, Vict_Age, Vict_Sex, Vict_Descent, DR_NO -> FIR)
Accused(AccusedID, Name, Age, Gender, Address, ArrestDate, DR_NO -> FIR)
Court(CourtID, CourtName, Address, Jurisdiction)
CourtCase(CaseID, CaseNo, FilingDate, Verdict, VerdictDate,
          DR_NO -> FIR [UNIQUE], CourtID -> Court, AccusedID -> Accused)
```

At this point, every column in every table depends only on that table's own key, and no data is stored twice — station names, premises names, weapon names, and status text now each live in exactly one place.

What I learned: Normalization made a lot more sense once I stopped thinking of it as a set of rules to memorize and started asking one simple question at each step: does this column really depend on the whole key, or is it hiding a smaller table inside this one? Splitting one large table into FIR, Victim, Accused, Officer, Court, and the lookup tables removed a lot of repeated data and made the database much easier to update correctly — for example, updating a police station's name in one place instead of in every single FIR row.

10. Indexing

To keep lookups fast, I used the following basic indexes, which are enough for a project database of this size:

Table	Index	Why
FIR	Primary Key on DR_NO	So every FIR record can be found quickly and stays unique
Officer, Victim, Accused, Court, CourtCase, etc.	Primary Key on each ID column	So every record in every table has a fast, unique way to be found
FIR, Officer, Victim, Accused, CourtCase, FIR_CrimeType	Foreign Key indexes (AREA, OfficerID, DR_NO, CourtID, Crm_Cd, etc.)	So joining tables together (e.g., FIR with Officer, or FIR with Victim) is fast
FIR_CrimeType	Index on Crm_Cd	So all FIRs of a particular crime type can be looked up quickly

What I learned: I originally wrote a much bigger indexing section with spatial and composite indexes, but for a database this size, Primary Key and Foreign Key indexes already cover the common lookups — so I kept this section simple instead of adding indexing techniques I wasn't using.

11. Sample SQL Queries

A few example queries to show how the tables work together once the database is set up:

11.1 List every crime type attached to a given FIR

```
SELECT f.DR_NO, c.Crm_Cd_Desc
FROM FIR f
JOIN FIR_CrimeType fc ON f.DR_NO = fc.DR_NO
JOIN CrimeType c ON fc.Crm_Cd = c.Crm_Cd
WHERE f.DR_NO = '211507896';
```

11.2 Monthly crime count per Police Station

```
SELECT p.AREA_NAME, MONTH(f.Date_Occ) AS month, COUNT(*) AS incidents
FROM FIR f
JOIN PoliceStation p ON f.AREA = p.AREA
WHERE YEAR(f.Date_Occ) = 2020
GROUP BY p.AREA_NAME, MONTH(f.Date_Occ)
ORDER BY p.AREA_NAME, month;
```

11.3 Accused persons awaiting a court verdict

```
SELECT a.Name, f.DR_NO, cc.CaseNo, co.CourtName
FROM Accused a
JOIN FIR f ON a.DR_NO = f.DR_NO
JOIN CourtCase cc ON cc.DR_NO = f.DR_NO
JOIN Court co ON cc.CourtID = co.CourtID
WHERE cc.Verdict IS NULL;
```

11.4 Officer-wise open investigation count

```
SELECT o.Name, o.BadgeNo, COUNT(*) AS open_cases
FROM Officer o
JOIN FIR f ON f.OfficerID = o.OfficerID
JOIN CaseStatus s ON f.Status = s.Status
WHERE s.Status_Desc = 'Invest Cont'
GROUP BY o.Name, o.BadgeNo
ORDER BY open_cases DESC;
```

What I learned: Writing these queries was actually a good way to double-check my schema — if a query needed a JOIN I couldn't easily write, it usually meant a foreign key was missing or a relationship was set up wrong.

12. Challenges Faced

A few things took extra time to figure out while designing this database:

- Deciding how FIR, Victim, and Accused should connect to each other. At first I tried to put victim and accused details directly inside the FIR table, but that broke down as soon as an FIR had more than one victim. I fixed this by giving Victim and Accused their own tables, each linked back to FIR through DR_NO.
- Working out where AREA_NAME really belonged. It looks like it belongs to the FIR row since it's right there in the dataset, but it actually depends on AREA, not on DR_NO — so it had to be moved out into its own PoliceStation table.
- Deciding how to handle the four repeating Crm_Cd columns. Simply keeping four columns felt wrong, so I looked at what 1NF actually required and switched to a junction table (FIR_CrimeType) instead.
- Adding the Accused, Officer, and Court tables, since these aren't columns in the raw dataset at all. I had to think through the full FIR-to-court workflow myself rather than just reading it off the dataset.

What I learned: Most of my confusion was really about functional dependencies — figuring out exactly what each column depends on. Once I got used to asking that question for every column, the rest of the normalization process became much more straightforward.

13. Implementation Details

This is how the design in this report would actually be built and tested:

Item	Detail
RDBMS	MySQL 8.0
Tool used	MySQL Workbench 8.0 (for designing and running the schema)
Number of tables created	12 (FIR, Accused, Victim, Officer, Court, CourtCase, CrimeType, FIR_CrimeType, PoliceStation, Premises, Weapon, CaseStatus)
Sample data inserted	About 20 sample FIR records (based on the LAPD dataset rows) to test the joins and queries in Section 11
Operating System	Windows 11

14. Conclusion

This project took a single, repetitive 28-column crime file and turned it into a clean, twelve-table database. Splitting the data removed the repeated crime-code columns, took station/premises/weapon/status text out of the FIR table, and added Accused, Officer, and Court tables so a case can be followed from the moment it's reported all the way to a court verdict. The result is a schema with far less duplicate data, clear primary/foreign key relationships, and simple indexes that keep the common lookups fast — a good, working example of why normalization matters in a real database.

15. References

- [1] City of Los Angeles Open Data Portal. Crime Data from 2020 to Present (Dataset ID: 2nrs-mtv8). LAPD Information Technology Bureau, Los Angeles, CA.
<https://data.lacity.org/Public-Safety/Crime-Data-from-2020-to-Present/2nrs-mtv8>
- [2] Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems (7th ed.). Pearson Education, Inc. [Chapter 9: ER-to-Relational Mapping; Chapter 14: Normalization]
- [3] Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts. McGraw-Hill Education.
- [4] Oracle Corporation. MySQL 8.0 Reference Manual.
<https://dev.mysql.com/doc/refman/8.0/en/>